# One Pricer a Day

Neel Dev Sen

July 2026

# Contents

# 1 Introduction

This is a project where I develop derivative pricers using **C++**. My goal is to do at least one a day and you can access all of my Github Repo

# 2 European Options

## 2.1 Day 1: Black Scholes

The first option that I am going to be pricing is the European option using the closed-form Black-Scholes equation.

In **Github** the full **C++** script is on this link: Day1-BlackScholesPricer

### 2.1.1 The Black Scholes PDE

The Black Scholes equation is a partial differential equation that is used to model how option prices change as the initial stock price, time to maturity, risk-free interest rate and implied volatility change. The partial differential equation is shown below:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{2.1}$$

In this equation $V$ is the option's price, $\sigma$ is the implied volatility, $r$ is the risk-free rate of interest and $t$ is the time to maturity.

$\frac{\partial V}{\partial t}$ is the rate of change of the option's price to time, $\frac{\partial V}{\partial S}$ is the rate of change of the option's price to the underlying stock price and $\frac{\partial^2 V}{\partial S^2}$ is a concavity factor on how the option price changes with the stock price. When solving the Black-Scholes there are two cases: when the option is a call option, or when the option is a put option.

### 2.1.2 Closed-Form Solution for European Call Options

The closed-form solution for the Black-Scholes equation where the option is a call option is:

$$C(t, S) = S\Phi(d_1) - Ke^{-r(T-t)}\Phi(d_2) \tag{2.2}$$

where:

$$d_1 = \frac{\ln(\frac{S}{K}) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}}$$

and:

$$d_2 = d_1 - \sigma\sqrt{T-t}$$

In this equation $T - t$ represents the time to maturity and $K$ represents the strike price.

$\Phi(z)$ is the standard normal cumulative distribution function which can be represented with the error function $\text{erf}(z)$:

$$\Phi(z) = \frac{1}{2}(1 + \text{erf}(\frac{z}{\sqrt{2}})) \tag{2.3}$$

In C++ this can be implemented as:

```
1  #include <iostream>
2  #include <cmath>
3  #include <type_traits>
4
5  template <typename T>
6  auto normalCDF(T x) -> std::common_type_t <double , T>
7  {
8          using commonType = std::common_type_t<double, T>;
9          return static_cast<commonType>(0.5) * (static_cast<
               commonType>(1) + std::erf(x / std::sqrt(2.0)));
10 }
```

The function we are creating is called **normalCDF**.This listing uses the **cmath** header file in order to import **std::erf**. It also uses a function template with a type placeholder **T** so that we can use this function for any type given. In order to ensure that this function has at least a type **double** we use the header file **type traits** which allows us to specify that we want this function to be at least double in line 6 trailing return type. In line 8 we ensure that the **commonType** alias used amongst all the literals and variables in this function are also at least of type **double**. This function returns the result shown in equation 2.3 with a type of at least **double**.

Now to implement the function shown in equation 2.2 in C++ we use:

```
1  template <typename SType, typename KType, typename rType,
       typename sigmaType, typename tType>
2  auto blackScholesCall(SType S, KType K, rType r, sigmaType
       sigma, tType t) -> std::common_type_t<double, SType, KType
       , rType, sigmaType, tType>
3  {
4          using commonType = std::common_type_t<double, SType,
               KType, rType, sigmaType, tType>;
5          auto S_new {static_cast<commonType>(S)};
6          auto K_new {static_cast<commonType>(K)};
7          auto r_new {static_cast<commonType>(r)};
8          auto sigma_new {static_cast<commonType>(sigma)};
9          auto t_new {static_cast<commonType>(t)};
10
11
12         auto d1 {(std::log(S_new / K_new) + (r_new + 0.5 *
               sigma_new * sigma_new) * t_new) / (sigma_new * std
               ::sqrt(t_new))};
13         auto d2 {d1 - sigma_new * std::sqrt(t_new)};
14
15
16         auto C {S_new * normalCDF(d1) - K_new * std::exp(-
               r_new * t_new) * normalCDF(d2)};
17         return C;
18         }
```

The function in this listing is called **blackScholesCall**. This listing again uses the same header files and the same principles as the one before and converts every thing into a **commonType** of at least type **double**. The formula for $d_1$ is shown in line 12 and the formula for $d_2$ is shown in line 13. Finally the result in equation 2.2 is shown in line 16 using the previously defined function **normalCDF**. The function returns the value of the **call option** with at least type **double**.

### 2.1.3 Closed-Form Solution for European Put Options

The closed-form solution for the Black-Scholes equation where the option is a put option is:

$$P(t, S) = Ke^{-r(T-t)}\Phi(-d_2) - S\Phi(-d_1) \tag{2.4}$$

using the same definitions of $d_1$ and $d_2$ in 2.2
The implementation of this in C++ is:

```
1  template <typename SType, typename KType, typename rType,
       typename sigmaType, typename tType>
2  auto blackScholesPut(SType S, KType K, rType r, sigmaType
       sigma, tType t) -> std::common_type_t<double, SType, KType,
        rType, sigmaType, tType>
3  {
4          using commonType = std::common_type_t<double, SType,
               KType, rType, sigmaType, tType>;
5          auto S_new {static_cast<commonType>(S)};
6          auto K_new {static_cast<commonType>(K)};
7          auto r_new {static_cast<commonType>(r)};
8          auto sigma_new {static_cast<commonType>(sigma)};
9          auto t_new {static_cast<commonType>(t)};
10
11
12         auto d1 {(std::log(S_new / K_new) + (r_new + 0.5 *
               sigma_new * sigma_new) * t_new) / (sigma_new * std
               ::sqrt(t_new))};
13         auto d2 {d1 - sigma_new * std::sqrt(t_new)};
14
15
16         auto P {K_new * std::exp(-r_new * t_new) * normalCDF(-
               d2) - S_new * normalCDF(-d1)};
17         return P;
18 }
```

The function in this listing is called **blackScholesPut**. This listing is almost identical to the one for **blackScholesCall**. The formula for $d_1$ is again shown in line 12 and the formula for $d_2$ is shown in line 13. Finally the result in equation 2.4 is shown in line 16 using the function **normalCDF**. The function returns the value of the **put option** with at least type **double**.